VolterraDSPPredictor

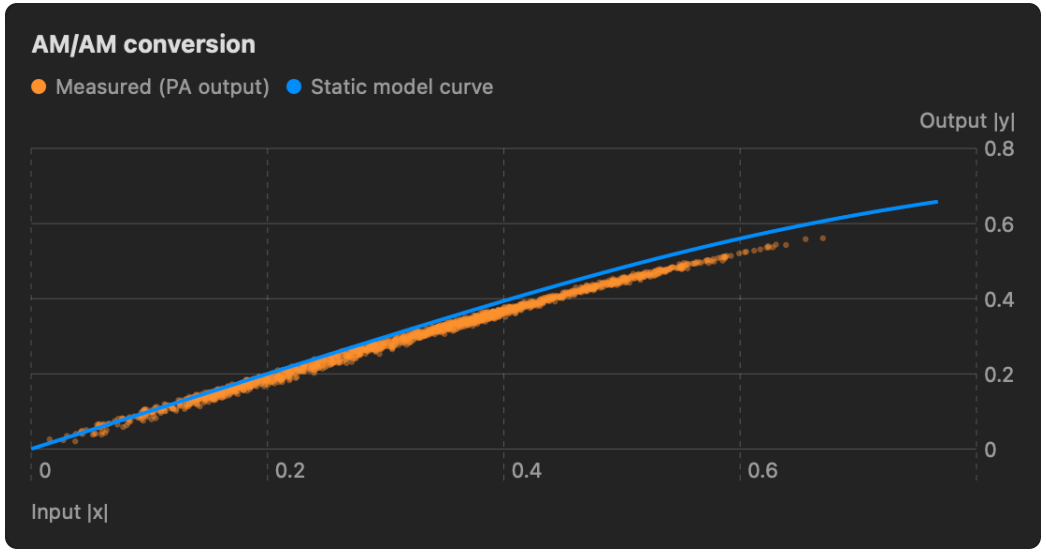
Teaching a radio amplifier to stay clean, using software that learns

A real-time simulator for adaptive digital predistortion on Apple Silicon

Boniface Mundia · October 2026 · github.com/BonnieMundia/VolterraDSPPredictor

The problem: amplifiers distort when pushed hard

- Every phone mast, Wi-Fi router and phone has a power amplifier that boosts the signal before the antenna
- Like a speaker turned up too loud, it distorts near its limit: the loudest moments get squashed and twisted
- It also has memory: what it does now depends on what it just did
- Result: errors in our own signal and noise that spills into other people's channels



Measured output (orange) falls below the ideal straight line as the input gets louder

Why it matters

Energy

The amplifier is the biggest power user in a radio. It is most efficient near its limit, which is exactly where it distorts.

Neighbours

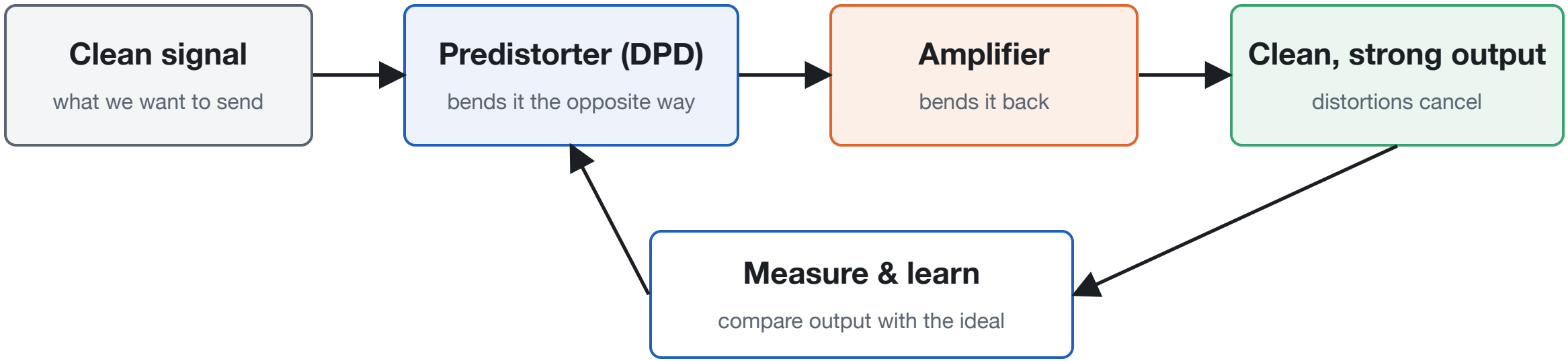
Distortion leaks into neighbouring frequency channels. Regulators set strict limits on this leakage.

Quality

A distorted signal causes bit errors, so users get slower, less reliable data.

Today's common fix is to turn the volume down ("back-off"). It works, but it wastes energy and money on every transmitter.

The idea: pre-distort the signal on purpose



- Like glasses: the lens bends light the opposite way your eye does, so you end up seeing clearly
- The amplifier changes with heat and age, so the predistorter keeps learning while it works

The gap this project addresses

- Most research is done offline: record data, compute, plot. You can't watch the learning happen or change things live.
- The most accurate model (the full Volterra series) is often dismissed as too slow. Few tools measure what it really costs.
- Modern chips where the CPU and GPU share memory, like Apple Silicon, are rarely studied for this job.
- Learning methods are compared in different papers with different setups, so the comparison isn't fair.
- Few open, reproducible tools exist for teaching: commercial toolboxes are closed and lab hardware is expensive.

What we built



The Live Dashboard updates ten times a second

- A Mac app that simulates the whole loop live: signal, predistorter, amplifier, learning, measurement
- Change any setting while it runs and see the effect instantly
- Four tabs: signal & amplifier, model engine, learning lab, live dashboard
- Written in Swift on Apple's Accelerate and Metal, with no external libraries and a 66-case test suite

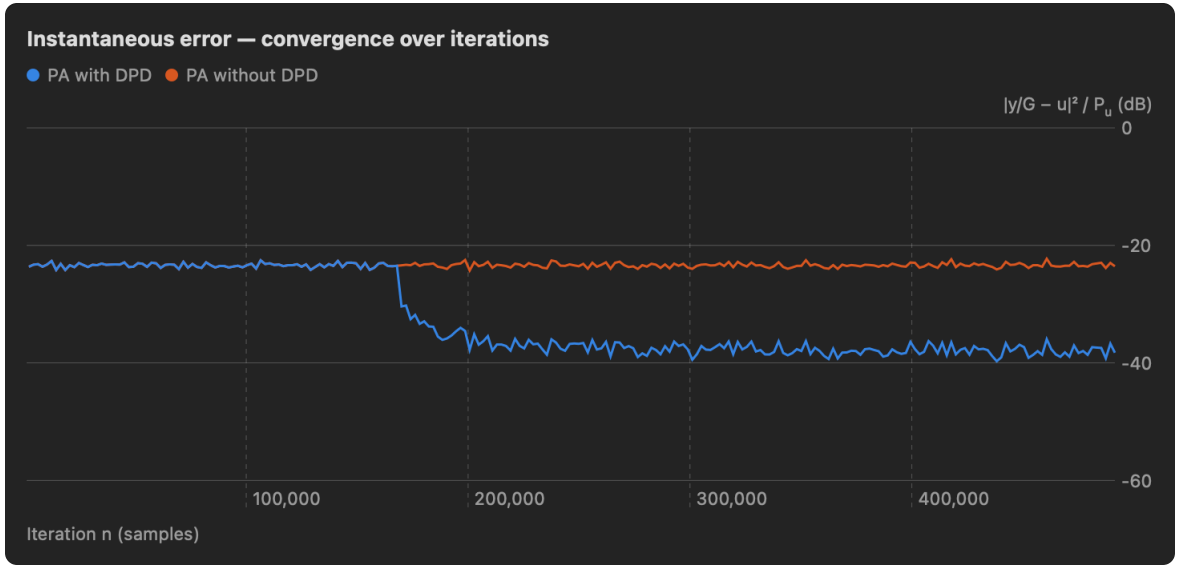
How it learns

A model with memory

The Volterra model looks at the last few samples and how they combine, not just the current one.

Three ways to learn its settings

- NLMS: small, cheap steps. Simple, but slow to settle.
- RLS: smart steps. Fast to settle, but heavy maths on every sample.
- Block RLS: the same smart answer, computed in batches. Fast and accurate, and the default.



Switching the predistorter on: the error (blue) drops about 15 dB below the uncorrected amplifier (orange) and stays there

Results: a much cleaner signal

23× less

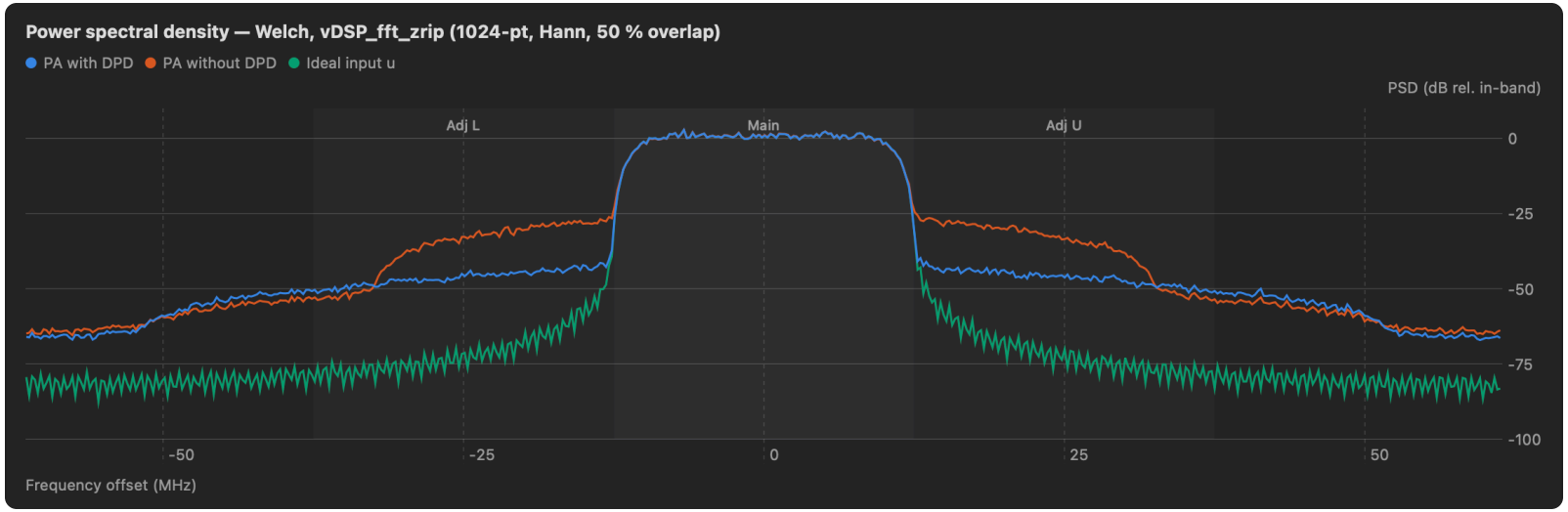
leakage to neighbours
–31.7 → –45.3 dBc

40× less

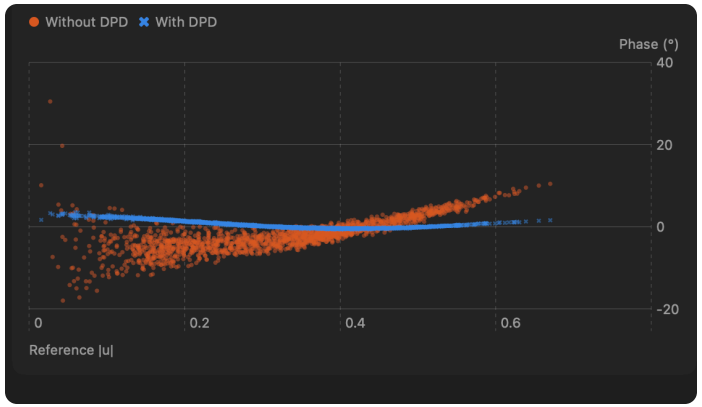
error in our own signal
–23.4 → –39.4 dB

7× lower

symbol error (EVM)
4.9 % → 0.7 %



Spectrum: orange = amplifier alone, blue = with predistortion, green = ideal



The twist is gone too: with predistortion the amplifier's phase error (blue) stays flat at 0°. Without it (orange) it wanders by 10–15°.

Making it fast

23× faster

Block RLS versus classic RLS (M = 8), with exactly the same result

26× faster

Keep only the important model terms and learn on 1 frame in 8. Same leakage result.

Auto CPU / GPU

The app times both processors while it runs and picks the faster one. The GPU wins for big models.

- All numbers measured on an Apple M5 laptop chip (Release build, 4,096-sample frames)
- Lesson: the "too slow" full model becomes practical with smart maths, not just faster hardware

Limits, next steps, and try it yourself

Honest limits

- Simulated amplifier, not real hardware yet
- Third-order model only (no 5th/7th order)
- Not yet at full radio speed (122.88 MS/s)

Next steps

- Test on a real amplifier with a software radio
- Higher-order and neural-network models

Try it

github.com/BonnieMundia/VolterraDSPPredictor
Open in Xcode 27 on an Apple Silicon Mac, then press ⌘R

Thank you. Questions?